```
"""Data Transform"""

from google.colab import files
uploaded = files.upload()

import pandas as pd
df = pd.read_csv("employee_productivity_dataset.csv")
pd.set_option('display.max_columns', None)
df.head(2)


"""### Data Overview and Missing Values"""

print(df.info())

print(df.describe(include='all'))

print(df.isnull().sum())


# Question 1: Handling Missing Values
# We are filling missing values for numerical columns using median and
mean as specified.
df['Age'] = df['Age'].fillna(df['Age'].median())
df['Salary'] = df['Salary'].fillna(df['Salary'].mean())
df['Hours_Worked_Per_Week'] =
df['Hours_Worked_Per_Week'].fillna(df['Hours_Worked_Per_Week'].median())
df['Performance_Score'] =
df['Performance_Score'].fillna(df['Performance_Score'].mean())

print("Missing values after imputation:")
print(df[['Age', 'Salary', 'Hours_Worked_Per_Week',
'Performance_Score']].isnull().sum())
display(df.head())


# Question.2: Label Encoding
# Label encoding converts categorical text data into numerical values.
Each unique category is assigned an integer
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

# Apply Label Encoding
df['Gender'] = le.fit_transform(df['Gender'])
df['Department'] = le.fit_transform(df['Department'])

print("Encoded columns (Gender & Department):")
display(df[['Gender', 'Department']].head())
display(df.head())


# Question 3: One-Hot Encoding
```

```
# One-hot encoding creates separate columns for each unique category in a
feature.
# This is useful for nominal categorical data where no inherent order
exists.
cols_before = df.shape[1]
df = pd.get_dummies(df, columns=['Work_Mode', 'Location'])
cols_after = df.shape[1]

print(f"Number of columns before One-Hot Encoding: {cols_before}")
print(f"Number of columns after One-Hot Encoding: {cols_after}")
print(f"New columns created: {cols_after - cols_before}")
display(df.head())


# Question 4: Normalization (Min-Max Scaling)
# Normalization scales numerical features to a fixed range, usually 0 to
1, without distorting differences in the ranges of values.
from sklearn.preprocessing import MinMaxScaler

mms = MinMaxScaler()

# Normalize Salary and Hours_Worked_Per_Week
df[['Salary', 'Hours_Worked_Per_Week']] = mms.fit_transform(df[['Salary',
'Hours_Worked_Per_Week']])

print("Normalized columns (Salary & Hours_Worked_Per_Week):")
display(df[['Salary', 'Hours_Worked_Per_Week']].head())
display(df.head())


# Question 5: Standardization (Standard Scaling)
# Standardization transforms data to have a mean of 0 and a standard
deviation of 1. It is less affected by outliers compared to Min-Max
scaling.
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

# Standardize Age and Projects_Completed
df[['Age', 'Projects_Completed']] = scaler.fit_transform(df[['Age',
'Projects_Completed']])

print("Standardized columns (Age & Projects_Completed):")
display(df[['Age', 'Projects_Completed']].head())
display(df.head())


# Question 6: Comparison of Scaling Methods (Salary Column)
# Normalization (Min-Max) maps values to [0, 1], while Standardization
(StandardScaler) centers the data around a mean of 0 with a unit standard
deviation.
import pandas as pd
from sklearn.preprocessing import MinMaxScaler, StandardScaler
```

```
# Reloading fresh data for comparison to ensure we show the effect on
original values
temp_df = pd.read_csv('employee_productivity_dataset.csv')
temp_df['Salary'] = temp_df['Salary'].fillna(temp_df['Salary'].mean())

mms_compare = MinMaxScaler()
ss_compare = StandardScaler()

comparison_df = pd.DataFrame({
    'Original_Salary': temp_df['Salary'],
    'MinMax_Scaled':
mms_compare.fit_transform(temp_df[['Salary']]).flatten(),
    'Standard_Scaled':
ss_compare.fit_transform(temp_df[['Salary']]).flatten()
})

print("Side-by-side comparison of scaling methods on Salary:")
display(comparison_df.head(10))


# Question 7: Building a Preprocessing Pipeline
# Using ColumnTransformer and Pipeline allows us to apply different
transformations to different columns simultaneously and bundle them into
a single object.
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer

# Reload fresh data for the pipeline
raw_df = pd.read_csv('employee_productivity_dataset.csv')

# Define features
numeric_features = ['Age', 'Salary', 'Hours_Worked_Per_Week',
'Projects_Completed', 'Performance_Score']
categorical_features = ['Gender', 'Department', 'Work_Mode', 'Location']

# Numeric Transformer: Impute then Scale
numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

# Categorical Transformer: One-Hot Encode
categorical_transformer = Pipeline(steps=[
    ('onehot', OneHotEncoder(handle_unknown='ignore'))
])

# Combine into a ColumnTransformer
preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numeric_features),
        ('cat', categorical_transformer, categorical_features)
    ])
```

```
# Apply the transformation
data_prepared = preprocessor.fit_transform(raw_df)

print("Shape of prepared data:", data_prepared.shape)
print("Preprocessing Pipeline constructed and applied successfully.")
```

 # Question 8: Applying and Displaying the Pipeline Results

```
# After fitting the pipeline, we transform the raw data into a numerical
matrix suitable for machine learning models. We'll examine the first few
entries and the final output shape.
import numpy as np

# The transformation was fit and applied in the previous cell as
'data_prepared'
print(f"Final Dataset Shape: {data_prepared.shape}")
print("\nFirst 5 rows of the transformed data (Head):")
# Displaying as a rounded array for better readability
print(np.round(data_prepared[:5], 3))
```

”””#Question 9:

Why is scaling important in Python (Machine Learning)?

Feature scaling is a critical step in preprocessing for several reasons:

1. **Algorithm Requirement**: Algorithms that calculate the distance between data points (like K-Nearest Neighbors, SVM, and K-Means clustering) are sensitive to the magnitude of features. If one feature has a range of 0-1 and another has a range of 0-1,000,000, the larger feature will dominate the distance calculation.
2. **Gradient Descent Convergence**: For algorithms using Gradient Descent (like Linear Regression, Logistic Regression, and Neural Networks), scaling helps the optimizer reach the global minimum faster by making the cost function contours more spherical rather than elongated.
3. **Comparability**: It allows us to compare coefficients in linear models directly, as all features are brought to a similar scale.

 # Question 10:

Why do we convert categorical data into numerical form?

Machine learning models are essentially mathematical functions that perform operations on matrices of numbers. We convert categorical data for the following reasons:

1. **Mathematical Compatibility**: Most algorithms (like Linear Regression, SVMs, and Neural Networks) cannot directly perform arithmetic operations on strings like "Remote" or "IT". They require numerical inputs to calculate gradients, distances, or weights.
2. **Feature Representation**: Encoding techniques like One-Hot Encoding allow the model to treat categories as distinct features without implying a false mathematical relationship (e.g., suggesting that "IT" is 'greater' than "HR").

3. **Standardization**: Converting everything to numbers allows categorical data to be treated with the same statistical rigor as numerical data, enabling the use of loss functions and optimization techniques.
"""